Lecture 4 · Part I · Lecture notes

Training models

Dataset: Titanic, 891 passengers

How to use these notes

The primary text for this lecture is Géron, Chapter 4. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

---

1 Why a closed form is not enough

Lecture 2 derived $\hat{\theta} = (\mathbf{X}^\intercal\mathbf{X})^{-1}\mathbf{X}^\intercal y$, valid exactly when $\mathbf{X}$ has full column rank. Lecture 3 measured a classifier properly. Both lectures called `.fit()` and moved on. Today we open it.

Situation	What the normal equation does
Many features	inverts an $(n{+}1) \times (n{+}1)$ matrix — cost grows like n^3
Many instances	forms $\mathbf{X}^\intercal\mathbf{X}$, so every row must be seen at once
Rank-deficient design	the inverse does not exist
A cost that is not squared error	there is no closed form at all

The last row is the one that matters. The classifier we build today minimises a cost whose stationary equations have no solution in elementary functions, so the answer has to be *walked to*.

2 Derivation: gradient descent

You have a cost $J(\boldsymbol{\theta})$ and no formula for its minimiser. The answer must supply three things: a direction to move in, a step size that does not overshoot, and a reason to believe the point you stop at is the minimum rather than merely a place where you stopped.

Throughout: m instances, n features, $\mathbf{X}$ of shape $m \times (n{+}1)$ with a leading column of ones, $\eta > 0$ the **learning rate**, and

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) = \frac{1}{m}\|\mathbf{X}\boldsymbol{\theta} - y\|_2^2.$$

We derive on squared error because it is the one cost whose answer we already have in closed form — which makes it the one cost where the method can be checked against a formula.

2.1 Which way is downhill

Move a small distance in a unit direction $\mathbf{u}$. To first order,

$$J(\boldsymbol{\theta} + \epsilon\mathbf{u}) - J(\boldsymbol{\theta}) = \epsilon\,\nabla J(\boldsymbol{\theta}) \cdot \mathbf{u} + o(\epsilon).$$

By Cauchy–Schwarz, $|\nabla J \cdot \mathbf{u}| \le \|\nabla J\|$ for every unit $\mathbf{u}$, with equality only when $\mathbf{u}$ is parallel to ∇J. So

$$\mathbf{u}^* = -\frac{\nabla J(\boldsymbol{\theta})}{\|\nabla J(\boldsymbol{\theta})\|}.$$

That is why the gradient and not some other vector: it is the *unique* direction of steepest decrease, and the argument is three lines long.

2.2 The gradient, and the update

One partial derivative, then stack them

$$\begin{aligned}\frac{\partial J}{\partial \theta_j} &= \frac{1}{m}\sum_{i=1}^{m} 2\big(\boldsymbol{\theta}^\top\mathbf{x}^{(i)} - y^{(i)}\big)\frac{\partial}{\partial \theta_j}\big(\boldsymbol{\theta}^\top\mathbf{x}^{(i)}\big)\\ &= \frac{2}{m}\sum_{i=1}^{m}\big(\boldsymbol{\theta}^\top\mathbf{x}^{(i)} - y^{(i)}\big)x_j^{(i)},\end{aligned}$$

using $\partial(\boldsymbol{\theta}^\top\mathbf{x})/\partial\theta_j = x_j$, because $\sum_k \theta_k x_k$ contains θ_j exactly once. Read it as *error* $\times$ *feature*, averaged over the instances. Stacking the partials,

$$\nabla J(\boldsymbol{\theta}) = \frac{2}{m}\,\mathbf{X}^\top\big(\mathbf{X}\boldsymbol{\theta} - y\big),$$

and the update is

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta\,\nabla J\big(\boldsymbol{\theta}^{(k)}\big).$$

One matrix product over the whole training set, at cost $O(mn)$ per evaluation — linear in the rows, where the normal equation was cubic in the columns. No matrix is ever inverted. It is called **batch** gradient descent because every step uses the whole training set.

Note that η has units: it converts a gradient into a displacement in parameter space. That is why its right value depends on how the features are scaled.

2.3 Where it stops, and whether that is the minimum

The iteration stops moving exactly when the step is zero:

$$\nabla J(\hat{\boldsymbol{\theta}}) = \mathbf{0} \iff \mathbf{X}^\top \mathbf{X}\hat{\boldsymbol{\theta}} = \mathbf{X}^\top y.$$

Descent does not solve a different problem

That is the normal equation of Lecture 2. Descent solves the same system without ever forming $(\mathbf{X}^\top\mathbf{X})^{-1}$ — which is why it survives the rank-deficient case the closed form cannot even state.

The Hessian is constant, $\nabla^2 J = \frac{2}{m}\mathbf{X}^\top\mathbf{X}$, and positive semi-definite everywhere. So J is convex: no local minima, no plateaux other than the minimum itself, no saddle points. Descent cannot get stuck; it can only be slow.

Hold on to the word *convex*. The moment we leave linear models, in Lecture 9, every one of those guarantees goes.

2.4 How large may the step be

Write $H = \nabla^2 J$ and subtract the fixed point from the update:

$$\boldsymbol{\theta}^{(k+1)} - \hat{\boldsymbol{\theta}} = (\mathbf{I} - \eta H)\,(\boldsymbol{\theta}^{(k)} - \hat{\boldsymbol{\theta}}).$$

In the eigenbasis of H each coordinate of the error is multiplied by $(1 - \eta\lambda_i)$ every step, so every mode contracts if and only if

$$0 < \eta < \frac{2}{\lambda_{\max}}.$$

Because J is quadratic this is exact, not a local approximation.

Learning rate	Factor per step	What you see
$\eta \ll 2/\lambda_{\max}$	just under 1	the cost falls slowly, and is still falling when you stop
$\eta \approx 1/\lambda_{\max}$	small	the cost drops fast and flattens
$\eta > 2/\lambda_{\max}$	$\lvert 1 - \eta\lambda_i \rvert > 1$	the cost rises, oscillates, and overflows to `nan`

Failure condition — divergence is silent until it is loud

A diverging fit does not raise. It returns weights full of `inf` and a model that predicts one value everywhere. Plot the cost against the step number; it is the only cheap way to see which of the three rows you are in.

2.5 Why you must scale the features

The step count is governed by the slowest mode. With the best fixed η the error contracts per step by

$$\frac{\kappa - 1}{\kappa + 1}, \qquad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \operatorname{cond}(\mathbf{X}^\top\mathbf{X}).$$

$\kappa = 1$ is a circular bowl and one step reaches the bottom; $\kappa = 100$ is a long thin valley the path zig-zags across; and κ large because one column is in thousands and another in units is a *scaling accident*, entirely avoidable.

Standardising the columns is not cosmetic. It changes κ, and κ is what sets the number of steps.

2.6 One instance at a time

Each batch step costs $O(mn)$, unaffordable for large m. Draw an index i uniformly at random and use only that row:

$$\mathbf{g}^{(i)}(\boldsymbol{\theta}) = 2\big(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)}\big)\mathbf{x}^{(i)}, \qquad \mathbb{E}_i\big[\mathbf{g}^{(i)}\big] = \nabla J(\boldsymbol{\theta}).$$

The one-instance gradient is an *unbiased* estimate of the batch gradient, and that one line is what makes stochastic and mini-batch descent legitimate rather than merely cheap.

Failure condition — unbiased is not the same as right

The estimate's variance does not shrink as $\boldsymbol{\theta}$ approaches $\hat{\boldsymbol{\theta}}$: at the minimum the batch gradient is zero and the individual gradients are not. So the iterates bounce around the minimum forever, and the remedy is a learning schedule $\eta_k \to 0$.
Averaging over b rows is still unbiased, by the same one-line argument, and divides the variance by b.

Variant	Rows/step	Cost	Path
Batch	m	$O(mn)$	smooth, straight to the minimum
Mini-batch	b	$O(bn)$	slightly noisy, and vectorises on hardware
Stochastic	1	$O(n)$	very noisy, never settles without a schedule

The bouncing is not only a cost. On a non-convex surface — every network from Lecture 9 onwards — it is what lets the iterate leave a bad local minimum. Here, on a convex bowl, it is pure nuisance.

What the derivation buys you

That the gradient is error times feature, so a linear model fits without inverting anything; that convexity licenses trusting the answer from any starting point, and that this stops being true in Lecture 9; that a rising cost curve is a step-size fault rather than a bug; that scaling is part of the algorithm, not tidiness; and that mini-batches are principled.
And one more: the derivation never used the fact that the cost was squared error until the partial derivative. Swap the cost, redo that one step, and everything else stands. That is exactly what we do next.

3 Fitting by descent

	Normal equation / SVD	Gradient descent
Cost in m	linear	linear, per step
Cost in n	$n^{2.4}$ to n^3	linear
Out-of-core	no — needs all rows	yes, with mini-batches
Scaling required	no	yes
Hyperparameters	none	η, batch size, schedule, stopping rule
Other costs	squared error only	any differentiable cost

Few features and data that fits in memory: use the closed form, it has no knobs to get wrong. Many features, streaming data, or any cost that is not squared error: descent.

3.1 Curvature without leaving linearity

A straight line cannot fit a curve, but $\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$ is linear in $\boldsymbol{\theta}$ even though it is not linear in x. Everything derived above applies unchanged — the gradient, the convexity, the convergence condition — because all of them were statements about $\boldsymbol{\theta}$.

`PolynomialFeatures(degree=d)` also generates every cross product, which is how a linear model represents an interaction such as "third class and travelling alone". The column count is $\binom{n+d}{d}$, so "a bit more capacity" is never a small change: degree 6 on four numeric columns gives the model 227 weights to determine from 712 passengers, about three rows per weight.

4 The worked example: a probability, not a label

A marine safety planning unit wants an evacuation-assistance model: for each passenger profile, how likely that person is to get off unaided — and they need to explain the assignment afterwards.

What they asked for	What that forces
"how likely"	a *probability*, not a label
the number is used to allocate	it must be *calibrated*
"explain the assignment"	weights a human can read

Why requirement 1 is not a formatting preference

"This passenger survives" is not checkable on its own. "This passenger survives with probability 0.31" is: take everyone the model puts near 0.31 and count.

4.1 The data, and three kinds of hole

891 passengers, 12 columns. `PassengerId` is an index dressed as a feature: give it to a model and the model will find something in it.

Three columns have holes, and they get three different answers. `Cabin` is missing for 687 passengers — 77%, which is not a column with gaps but a gap with a column — so do not impute it: take the

deck letter where there is one and give the rest their own level `U`, because the missingness *is* the information. `Age` is missing for 177, imputed by median inside the pipeline so it is recomputed on every training fold. `Embarked` is missing for 2, imputed by the most frequent port; two rows out of 891 cannot matter, and you should still be able to say what you did.

Note that `Deck = "U"` will largely mean third class, because the cabin number was recorded for passengers whose tickets carried one. So the column is partly a proxy for `Pclass`. Write that down.

4.2 The label, and a column that looks like free text

Outcome	Passengers	Share
Did not survive	549	61.6%
Survived	342	38.4%

Mildly imbalanced — nothing like the rare-event problem of Lecture 3. Accuracy will not be absurd here. It will be something more dangerous than absurd: plausible.

Every name carries a *title*, and the title carries sex, marital status and — for *Master* — being a boy.

Title	Passengers	Survived
Mrs	126	79.4%
Miss	185	70.3%
Master — a boy	40	57.5%
Rare — Dr, Rev, Col, . . .	23	34.8%
Mr	517	15.7%

Master is the only level that is not a re-encoding of `Sex`: those 40 boys survived at nearly four times the rate of the men.

Four engineered columns, each with a reason: `Title` (a boy is not a man), `FamilySize` (a group evacuates together, and slowly), `IsAlone` (nobody is waiting for you), `Deck` (distance to a boat, and a proxy for class). Note that `FamilySize = SibSp + Parch + 1` is an exact linear function of two columns we already have. Remember it — we come back to it with a rank computation, not with an opinion.

4.3 Split first, and measure the anchor

Stratified on the *label* this time, not on a predictor: with 179 test rows an unstratified draw can shift the base rate by several points, and every downstream number moves with it. That gives 712 training and 179 test passengers, at rates 0.3834 and 0.3855.

With $p = 0.3834$, a model that has learned nothing except the base rate scores $1 - p = 0.617$ accuracy and a log loss of

$$-\big(p \log p + (1 - p) \log(1 - p)\big) = 0.666$$

— the entropy of the label, in nats. Every number below has to be compared against it.

5 Logistic regression

5.1 The model, and what the linear part predicts

$$\hat{p} = \sigma\big(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}\big), \qquad \sigma(t) = \frac{1}{1+e^{-t}}.$$

$\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}$ is exactly the quantity least squares fitted; everything new is in σ and in what we ask $\boldsymbol{\theta}$ to minimise. σ is strictly increasing and maps $\mathbb{R}$ onto $(0, 1)$, so it never returns exactly 0 or 1 — which matters when the loss takes a logarithm of it.

Inverting it:

$$t = \log\frac{\hat{p}}{1-\hat{p}} = \boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}.$$

The sentence most often got wrong in the examination

The linear model does not predict the probability. It predicts the *log-odds*, and the coefficients act on the *odds*: add 1 to x_j and the odds are multiplied by e^{θ_j}, wherever you started. The probability has no such rule.

5.2 The loss, and why this one

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\Big[y^{(i)}\log\hat{p}^{(i)} + \big(1-y^{(i)}\big)\log\big(1-\hat{p}^{(i)}\big)\Big].$$

Only the probability given to the true class enters each term, and it is unbounded above: claim 0.001 for a passenger who survived and that term is $-\log 0.001 \approx 6.9$.

A scoring rule is **proper** if the expected score is optimised by reporting your true belief. Log loss is proper; so is the Brier score $\frac{1}{m}\sum(\hat{p}-y)^2$, which is bounded and easier to read, and we report it too.

Failure condition — accuracy is not proper

Under accuracy a model reporting 0.51 scores exactly the same as one reporting 0.99. So accuracy gives a model no reason at all to tell the truth about its uncertainty, and optimising it destroys calibration.

5.3 Its gradient, and the absence of a closed form

Redo the one step of the derivation that used the cost. $\sigma'(t) = \hat{p}(1-\hat{p})$, and $\partial\ell/\partial\hat{p} = (\hat{p}-y)/\hat{p}(1-\hat{p})$, so the two factors cancel exactly and

$$\frac{\partial J}{\partial\theta_j} = \frac{1}{m}\sum_{i}\big(\hat{p}^{(i)}-y^{(i)}\big)x_j^{(i)}.$$

Error times feature. The same shape as the gradient of the MSE.

And yet setting it to zero gives $\mathbf{X}^{\mathsf{T}}(\sigma(\mathbf{X}\boldsymbol{\theta})-y) = \mathbf{0}$, in which $\boldsymbol{\theta}$ appears inside a transcendental function; no rearrangement isolates it. But the Hessian is $\frac{1}{m}\mathbf{X}^{\mathsf{T}}\mathbf{S}\mathbf{X}$ with $\mathbf{S} = \operatorname{diag}(\hat{p}^{(i)}(1-\hat{p}^{(i)})) \succeq 0$, so the surface is a bowl and descent reaches the global minimum.

This is the first model in the course with no formula for its own answer. Everything in the first half of today is what makes it fittable anyway.

5.4 The first numbers

Fitted with `C=1e6`, which turns Scikit-learn's default penalty *off*, so today's coefficients are the data's rather than a compromise with a penalty. Turning it back on is the next lecture.

Model	Log loss	Accuracy
Report the base rate to everyone	0.666	61.7%
Logistic regression, 10-fold cross-validated	0.496	81.5%

About a quarter below the anchor on log loss and twenty points of accuracy above the majority class. And the per-fold spread of the log loss is 0.22 — larger than every difference we are about to compare. Carry that number.

6 Three coefficients that cannot be right

Feature	Coefficient	Odds multiplier
`Deck_T`	−5.678	×0.003
`Title_Master`	+3.484	×32.596
`Sex_female`	+3.308	×27.325
`Sex_male`	−2.833	×0.059
`Title_Miss`	−2.806	×0.060
`Title_Mrs`	−2.020	×0.133

`Title_Miss` at −2.81, when 70.3% of them survived against 38.4% overall. `Title_Mrs` at −2.02, the highest-surviving group in the data. And `Deck_T` is the largest weight in the model, when there is exactly one passenger on deck T.

These are not weak effects or noisy estimates. They have the wrong *sign*, and the model's predictions are nonetheless good. Before reaching for a story about confounding, ask the cheaper question: what is the shape of the design matrix?

6.1 Count the columns, then count the rank

```
print("columns:", Z_b.shape[1])            # 29
print("rank:   ", np.linalg.matrix_rank(Z_b))  # 23
print("cond:   ", np.linalg.cond(Z_b))         # 2.6e+16
```

Six columns are linear combinations of the others. Five come from the encoder — each one-hot block sums to 1 in every row, and the intercept column is also 1 in every row, so five categorical columns give five exact dependencies. The sixth is ours: `FamilySize = SibSp + Parch + 1`, exactly, for every passenger, with maximum residual 0.0. Not "highly correlated" — an

identity. Standardising does not help, because an affine map of a linear dependence is still a linear dependence.

Failure condition — what a rank-deficient design does to the fit

If $\mathbf{X}\mathbf{v} = \mathbf{0}$ for some $\mathbf{v} \neq \mathbf{0}$, then $\mathbf{X}(\boldsymbol{\theta} + c\mathbf{v}) = \mathbf{X}\boldsymbol{\theta}$ for every c: along $\mathbf{v}$ the loss is flat. There is not one best $\boldsymbol{\theta}$ but a whole affine subspace of them. The solver returns one point of it and says nothing. Predictions are unaffected; *every claim about a coefficient is void*.

Proved on our own matrix: add 2.5 to `Sex_female` and to `Sex_male`, subtract 2.5 from the intercept, and the largest change in any of the 712 logits is 1.8×10^{-15}. `Sex_male` can be -2.83 or -0.33, and both fits make the same prediction for every passenger.

The test, stated generally

If a coefficient can be moved without changing any prediction, that coefficient is not a property of the data.

6.2 The repair

Drop one level per categorical block and remove `FamilySize`:

Design matrix	Columns	Rank	Condition number
As first written	29	23	2.6×10^{16}
One level dropped, `FamilySize` removed	23	23	83.6

Fourteen orders of magnitude. `IsAlone` stays: it is an indicator, not a linear function of the counts.

Each dropped level becomes the **reference**, folded into the intercept, and every remaining coefficient is read *relative to it*. A coefficient quoted with no stated reference level cannot be read by anybody, including you in a month — and this is the most common defect in the regression tables you will be asked to review.

Use `min_frequency` rather than `handle_unknown="ignore"` here: with `drop="first"` an ignored unknown level is encoded all-zeros, which is the encoding of the reference level, so our one deck-T passenger would be scored as though they were on deck A whenever a fold holds them out.

Encoding	Cross-validated log loss
All levels, `FamilySize` kept — rank 23 of 29	0.496
One level dropped, `FamilySize` removed — full rank	0.468

What that table does and does not license

Not a large move, and not the point. The point is that the first row was produced by a fit that had no unique answer, and nothing said so. The fold spread of the repaired model is 0.13, and of the rank-deficient one 0.22 — both far wider than this gap.
Do not claim the second encoding is *better* on this evidence. Claim it is *defined*.

Now a coefficient can be read aloud. `Pclass_3` $= -1.169$: holding every other recorded feature fixed, travelling third class multiplies the odds of survival by $e^{-1.169} = 0.31$, relative to first class.

Every clause matters — "holding every other feature fixed" is doing real work, the multiplier is on odds rather than probability, and it is always relative to the reference level. And unique is not the same as stable: `Sex` and `Title` still carry much of the same information, so how the effect splits between them moves with the sample. We only fixed uniqueness.

7 Capacity, and the two ways a fit fails

A linear term in `Age` cannot say "children and the elderly were helped first", because that is not a monotone statement about age. Add Age^2: the log-odds become quadratic in age, the boundary becomes a conic, and the model is still linear in $\boldsymbol{\theta}$.

Degree	Columns	Training	Held-out	Held-out accuracy
1	22	0.395	0.468	81.5%
2	32	0.381	0.467	82.2%
3	52	0.355	0.538	79.6%
4	87	0.310	1.390	77.4%
5	143	0.286	1.922	76.0%
6	227	0.284	1.836	76.7%

Degrees 1 and 2 differ by 0.001 in held-out log loss, and the fold spread is over a hundred times that. Those two are a *tie*; reporting degree 2 as the winner would be reporting noise.

From degree 2 to degree 5 held-out accuracy falls by about six points while held-out log loss is multiplied by more than four. Accuracy only notices when a prediction crosses the cut-off; log loss notices a confident prediction becoming a confidently *wrong* one. The failure is not that the model gets more passengers wrong — it is that it becomes certain about the ones it gets wrong. The requirement was for probabilities, and the metric that matches the requirement is the metric that saw the damage.

7.1 A warning worth reading

Degree	1	2	3	4	5	6
Converged	yes	yes	yes	no	no	no
Iterations used	74	256	935	4000	4000	4000

The reflex is to raise `max_iter`. It cannot help, and understanding why is the point.

Failure condition — under separation the minimiser does not exist

With 87 columns and 712 rows the two classes eventually become linearly separable. If some $\boldsymbol{\theta}$ separates them perfectly, so that $\boldsymbol{\theta}^\top \mathbf{x}^{(i)}$ has the right sign for every i, then $J(c\,\boldsymbol{\theta}) \to 0$ as $c \to \infty$.

The likelihood has no maximum: scaling the weights up always improves it, and they run away to infinity. The warning is not a numerical nuisance — it is the solver reporting that the estimate you asked for *does not exist*.

8 Calibration, and the cut-off nobody chose

Definition 8.1 (Calibration)**.** $P(y = 1 \mid \hat{p}(\mathbf{x}) = q) = q$ *for every* $q \in (0, 1)$.

Take every passenger the model scored near q; the fraction who actually survived should be about q. This is a property of a *set* of predictions, not of any one prediction — which is exactly why it is so easy to ship a model that fails it.

Check it out of fold, with `cross_val_predict` rather than `predict`, so every probability came from a fit that had never seen that passenger. A reliability diagram drawn on training predictions always looks excellent.

Read the diagram by looking at the marker area first: a bin with six passengers has an observed rate that swings by a sixth on one person, so deviations in the middle bins are mostly sampling noise while deviations in the crowded end bins are not. Our expected calibration error, out of fold, is 0.050 — and it is roughly calibrated because log loss is a proper scoring rule and we minimised it directly. The calibration is not a happy accident; it is what the objective was asking for.

Failure condition — a bare label hides a decision nobody made

`.predict()` is `.predict_proba() >= 0.5`. That cut-off is cost-optimal only when the two mistakes cost the same. Nothing in the code mentions 0.5, nothing errors, and the accuracy printed is true — it just answers a question the stakeholder did not ask.

Measured rather than asserted, over twenty splits, choosing the cut-off on out-of-fold training predictions against a stated $10:1$ cost ratio:

Cut-off	Accuracy	Expected cost
0.50 — the library default	81.8%	169
0.86 — chosen from the costs, per split	74.1%	59

The default costs 110 escort-units more per evacuation on average, and it is worse on 20 splits out of 20. *Accuracy goes down when you do the right thing* — so if accuracy is the headline, you will reject the better rule.

For a calibrated model the cost-minimising cut-off is $t^* = c_{\mathrm{FN}}/(c_{\mathrm{FN}} + c_{\mathrm{FP}})$, which is 0.909 at a $10:1$ ratio. Our measured optimum sits below it, and that gap is the calibration error expressed in a unit somebody cares about.

So: fit and report on a proper scoring rule, both cross-validated; choose any cut-off on out-of-fold predictions against costs or a capacity somebody has stated; and report accuracy at that cut-off as a *consequence*, never as the objective.

9 More than two classes

Softmax regression gives one parameter vector per class:

$$s_k(\mathbf{x}) = \boldsymbol{\theta}_k^{\top}\mathbf{x}, \qquad \hat{p}_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^{K} \exp(s_j(\mathbf{x}))},$$

so the output is a distribution over the K classes by construction. It is multi*class*, not multi*label*: four objects in one photograph need K separate binary classifiers, not this.

Its cost is cross-entropy,

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log \hat{p}_k^{(i)},$$

whose inner sum keeps exactly one term. At $K = 2$ this is the log loss, term for term: logistic regression is the two-class case of softmax, not a different model. And its gradient is

$$\nabla_{\theta_k} J = \frac{1}{m}\sum_{i} (\hat{p}_k^{(i)} - y_k^{(i)})\mathbf{x}^{(i)}$$

— error times feature, for the third time today. That is not a coincidence; it is a property of this family of losses paired with these output functions, and it is why one optimiser fits all of them, including every network in the second half of this course.

The boundary between classes j and k is where $(\boldsymbol{\theta}_k - \boldsymbol{\theta}_j)^\top \mathbf{x} = 0$: a hyperplane, so the decision regions are convex polyhedra. Which is also the limitation — a class that is not linearly separable from the union of the others cannot be carved out, however many iterations you run.

10 What today's model is still missing

Left open	What it means	What fixes it
Rank 23 of 29 columns	the minimiser is not unique	a penalty makes it unique
lbfgs will not converge at degree 4	the minimiser does not exist	a penalty makes it exist
Held-out log loss quadruples from degree 2 to 5	capacity with no counterweight	a penalty buys most of it back
The degree was chosen by reading the held-out score	a choice with no diagnosis behind it	learning and validation curves

The first two are statements about the optimisation problem, not about the passengers. One idea repairs all four, and it is the next lecture.

11 The notebook

What to change

Gradient descent is implemented from the derivation in a dozen lines, checked against the normal equation to eight decimals, and run at three learning rates, one of which diverges. Raise `ETA_DIVERGE` and watch where the cost first becomes `inf`. Then compute $2/\lambda_{\max}$ for that design matrix and check the two agree — which turns the convergence condition from a claim on a page into a measurement you made.

12 Exercises

1. State the condition on η for batch gradient descent on the MSE to converge, and explain why the argument is exact rather than a local approximation. [5]

2. A design matrix has 29 columns and rank 23. A colleague reports that one coefficient is -2.83 and concludes the feature is protective. State what is wrong with the conclusion, and give a two-line computation that would demonstrate it. [5]

3. A logistic fit reports `ConvergenceWarning` at 4,000 iterations on a degree-4 polynomial expansion of 712 rows. Explain why raising `max_iter` cannot help. [4]

4. Held-out accuracy falls by six points between two models while held-out log loss quadruples. Explain what has happened to the predictions, and which metric you would report given a brief that asks for probabilities. [4]

5. A calibrated model is used with a $10:1$ cost ratio between false negatives and false positives. Give the cost-minimising cut-off and explain why the default 0.5 produces a *more accurate* and *more expensive* rule. [4]

Summary

Descent moves along $-\nabla J$, which Cauchy–Schwarz identifies as the unique steepest direction. On the MSE that gradient is $\frac{2}{m}\mathbf{X}^\top(\mathbf{X}\boldsymbol{\theta} - y)$ and its zero is the normal equation, so descent solves Lecture 2's system without ever forming an inverse. The cost is convex, so any stationary point is global; the step converges if and only if $\eta < 2/\lambda_{\max}$; and the step count is set by the condition number, which is why scaling is part of the algorithm. One row gives an unbiased gradient, which is what makes mini-batches legitimate.

Logistic regression predicts the log-odds linearly, minimises a proper scoring rule, and has no closed form — but its cost is convex, so the machinery above fits it. Its fit failed in two distinct ways here: *not unique*, because six of 29 columns were linear combinations of the others, proved by shifting two coefficients by 2.5 and moving no logit by more than 10^{-15}; and *non-existent* at degree 4, where separation sends the weights to infinity. Repairing the encoding took the condition number from 2.6×10^{16} to 83.6.

And a probability is checkable where a label is not. `.predict()` applies a cut-off of 0.5 that nobody chose; choosing it against a stated $10:1$ cost ratio was worse on accuracy, better on cost, on 20 splits out of 20.